



Part of the AgentCon World Tour by the Global AI Community

AI Agents Developer Conference

agentcon.city/rome



Designing Deterministic Agents Around Non-Deterministic LLMs

Sepehr Samadi

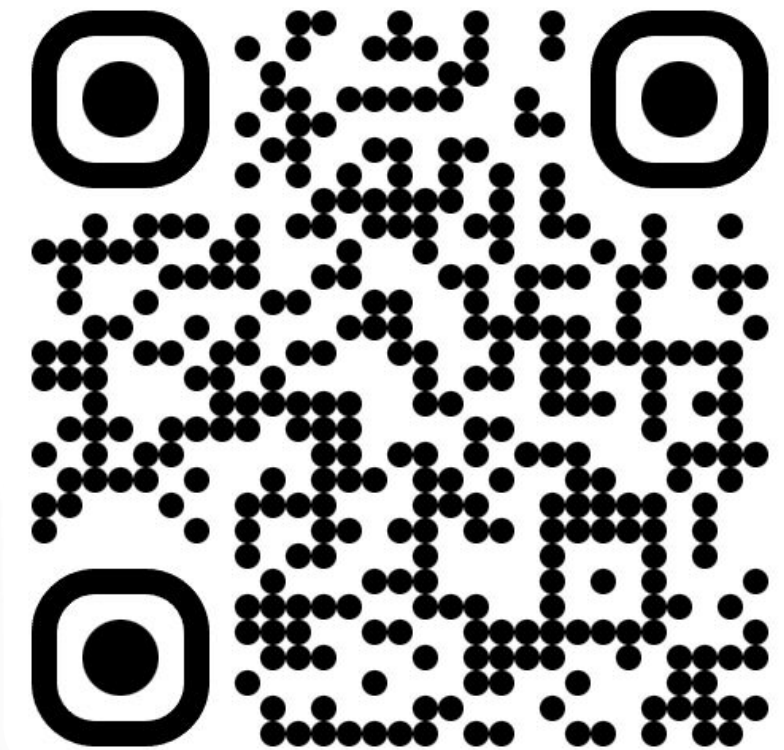


agentcon.city/rome

Made Possible by



Join the AI Conversation on Discord



gaic.io/discord

The Core Tension



LLMs Are Probabilistic

- Non-deterministic outputs
- Hallucinate with high confidence
- Drift over long contexts
- Fail silently in production



Production Is Deterministic

- Predictable, repeatable behaviour
- Contracts and schemas enforced
- Failures are observable and recoverable
- SRE principles apply

This session: engineering-first patterns to architect deterministic systems around non-deterministic models.

Four Pillars of Agent Resilience

1



Structured Contracts & Validation

Output schemas, guard layers, confidence scoring

2



Failure Mode Taxonomy

Tool misuse, hallucination, context drift, loops

3



Resilience Patterns

Retry/escalation, idempotency, token budgets, sandboxing

4



Observability

Tracing, logging, alerting, silent failure debugging



01

Structured Contracts & Validation

Enforcing determinism at the boundary between your agent and the outside world

Problem 0: Unstructured Output

Unstructured LLM Response

```
"Sure! The ticket seems urgent,
probably priority 4 or 5, and yes
I'd escalate it. The category
is billig."
```

- No schema
- No type safety
- Ambiguous values
- Typo in category ("billig")

What You Actually Need

```
{
  "priority": 4,
  "escalate": true,
  "category": "billing",
  "confidence": 0.87
}
```

- Typed fields
- Validated at the boundary
- Machine-readable contract

Every downstream system depends on structured, validated output.

Basic Solution: Enforcing Output Schemas

JSON Mode

```
response = client.chat
.completions.create(
    model="gpt-4o",
    response_format={
        "type": "json_object"
    }
)
```

Use when: Simple JSON needed

Pydantic Schema

```
class Response(BaseModel):
    priority: int
    escalate: bool

response = client.beta
.chat.completions.parse(
    model="gpt-4o",
    response_format=Response
)
```

Use when: Type safety + validation

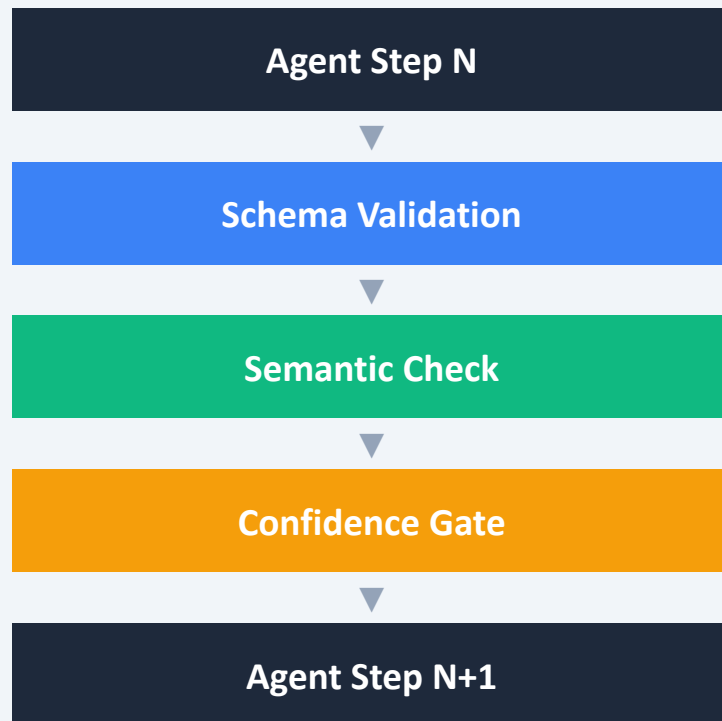
Function Calling

```
tools = [{
    "type": "function",
    "function": {
        "name": "analyze_ticket",
        "parameters": {
            "properties": {
                "priority": {
                    "type": "integer"
                }
            }
        }
    }
}]
```

Use when: Tool-calling workflows

Advanced: Semantic Guards & Confidence Scoring

Guard Layers Between Agent Steps



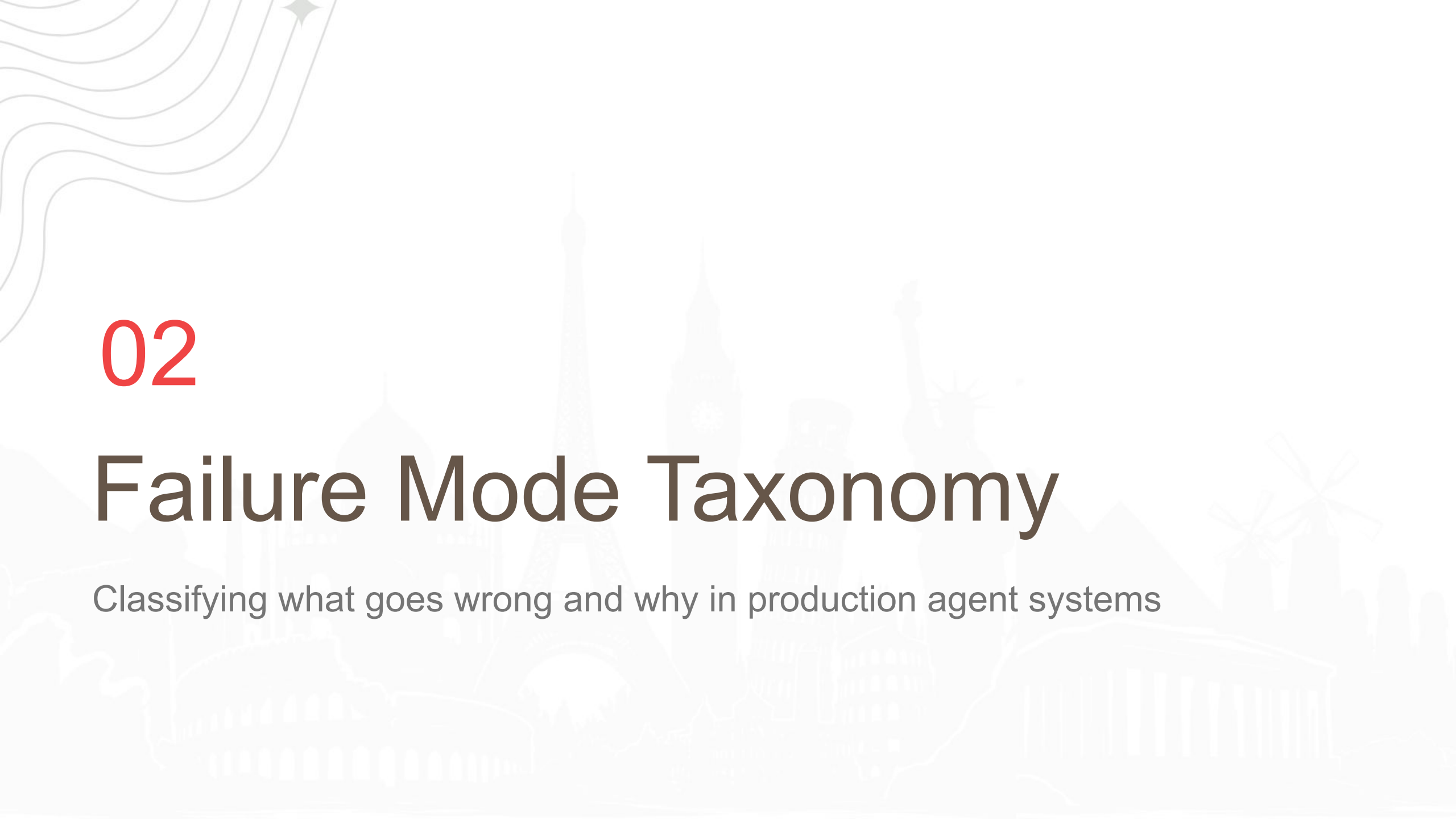
Confidence Scoring

```
# Confidence gate pattern
confidence = evaluate_response(
    response, context, kb
)

if confidence >= 0.9:
    return response # Auto-approve
elif confidence >= 0.6:
    escalate_to_human(response)
else:
    reject_and_retry(response)
```

When to trust your agent's answer:

- $\geq 0.9 \rightarrow$ Auto-approve
- $0.6-0.9 \rightarrow$ Human review
- $< 0.6 \rightarrow$ Reject and retry



02

Failure Mode Taxonomy

Classifying what goes wrong and why in production agent systems

Problem 1: Tool Misuse & Runaway Tool Loops

Real Failure Scenario

"Clean up all tickets from spam_bot@evil.com"

Agent calls delete_all_user_tickets() - 47 tickets gone, irreversible.

- Called a dangerous, irreversible tool
- No approval workflow enforced
- No sandbox. Executed directly in production
- No rollback or undo mechanism

Runaway Tool Loops

- Agent calls same tool repeatedly
- No loop detection
- Burns tokens endlessly
- Produces zero useful output



Real-World Equivalents

Deleted production database

Charged wrong customer
£10,000

Sent email to 50,000 users

Modified critical infra config

Basic Solution: Tool Schemas with Constraints

```
SAFE_TOOLS = [  
  {  
    "type": "function",  
    "function": {  
      "name": "read_ticket",  
      "description": "SAFE - read-only.  
        Use to inspect tickets.",  
      "parameters": {...}  
    }  
  },  
  {  
    "type": "function",  
    "function": {  
      "name": "mark_as_spam",  
      "description": "SAFE - reversible.  
        Does NOT delete.",  
      "parameters": {...}  
    }  
  }  
]  
# delete_all_user_tickets NOT included!
```

Benefits

- Agent can only call exposed tools
- Clear descriptions guide selection
- Simple to implement

Limitation

Agent could still call `mark_as_spam` 1,000 times in a loop, basic schemas alone don't prevent repetition.

90%

Success Rate

1.0×

Relative Cost

Advanced: Sandboxed Execution + Idempotency + Approval

```
class SandboxedToolExecutor:
    def execute(self, tool_name, params):
        # 1. Idempotency check
        key = hash(tool_name, params)
        if key in cache:
            return cache[key]
        # 2. Approval for dangerous tools
        if tool_name in DANGEROUS_TOOLS:
            if not request_approval(
                tool_name, params):
                raise PermissionError("Denied")
        # 3. Execute in sandbox
        result = run_in_sandbox(
            tool_name, params)
        # 4. Cache result
        cache[key] = result
        return result
```

Idempotency Cache

Same call twice → cached result. Safe to retry without duplicate side effects.

Approval Workflow

Slack/Teams notification with human approval gate for critical operations.

Sandbox Isolation

Container Apps with network isolation and resource limits per execution.

Feature	Basic	Advanced
Prevents wrong tool	✓	✓
Prevents repeated calls	✗	✓
Approval for dangerous ops	✗	✓
Safe to retry	✗	✓

Problem 2: Hallucination & Context Drift

Confident Hallucination

User cites fake policies, agent echoes them with confidence: 0.92

Section 4.2: DOESN'T EXIST
Article 9(b): DOESN'T EXIST
Policy 7.3.2: DOESN'T EXIST

Actual KB:
kb-001: Refunds 5–7 days
kb-003: SLA 4-hr response

In multi-agent graphs (AutoGen/CrewAI), one hallucinating agent poisons the entire chain.

Context Drift Over Long Runs

Turn 1 — Correct

"Refunds within 5-7 business days."

Turn 7 — Contradicts Turn 1

"Refunds issued immediately for Enterprise."

As the context window fills, older turns get truncated. The agent forgets its own earlier answers.

Basic Solution: LLM-as-Judge Validation

```
class ValidationResult(BaseModel):
    is_grounded: bool
    issues: List[str]
    reasoning: str

supervisor_prompt = f"""
Does this response cite policies
not present in the knowledge base?
KB: {kb_text}
Response: {agent_response}
Rules:
1. Every claim must trace to KB
2. No invented policy numbers
3. No meaning changes via paraphrase
"""

validation = supervisor_llm.validate(prompt)
if not validation.is_grounded:
    reject_response()
```

Strengths

- Catches semantic errors & paraphrasing
- No custom model required
- Simple to integrate

Limitations

- Judge itself can hallucinate
- Non-deterministic by nature
- Doesn't prevent context drift

~60-70%

Accuracy

2×

API Calls

Advanced: Semantic Entropy + Context Management

Part 1: Semantic Entropy

```
# Generate N=5 samples
samples = [llm.generate(prompt)
            for _ in range(5)]
# Measure disagreement
categories = [s.category for s in samples]
entropy = -sum(
    p * log(p)
    for p in Counter(categories).values()
)
# High entropy = hallucination risk
if entropy > THRESHOLD:
    flag_as_hallucination()
```

Part 2: Context Window Management

```
class ContextWindowManager:
    def get_managed_context(self):
        recent = conversation[-10:]
        older = conversation[:-10]
        summary = llm.summarize(older)
        return [system_prompt,
                summary] + recent

# Prevents context drift by
# preserving critical early turns
# while managing window limits
```

Metric	LLM Judge	Semantic Entropy
Hallucination Detection	~60-70%	~95%+
Context Drift Prevention	✗	✓
API Calls	2×	5-7×
Multi-turn Support	✗	✓

When to use: High-stakes domains (legal, medical, financial) and multi-turn conversational agents.



03

Resilience Patterns

Retry, reroute, or stop and knowing which to do when

Problem 3: Recursive Loops & Token Exhaustion

Recursive Reasoning Loop

"Research LLM reasoning advancements. Keep searching until you find definitive information."

- Agent interprets "keep searching" literally
- No loop detection or pattern recognition
- Repeated identical API calls waste budget
- Eventually hits timeout. Zero useful output



Token Budget Exhaustion

5-Step Market Analysis

- 01 Search competitor data
- 02 Analyse pricing
- 03 Identify market gaps
- 04 Generate recommendations
- 05 Executive summary

Each step: ~2,000 tokens
Total needed: 10,000
Budget: 8,000 tokens
SHORTFALL: 2,000 tokens

Basic Solution: Iteration Limits + Token Counting

```
class BasicWorkflowRunner:
    def __init__(self,
        max_iterations=10,
        max_tokens=10000):
        self.max_iterations = max_iterations
        self.max_tokens = max_tokens
        self.tokens_used = 0
        self.iterations = 0

    def execute_step(self, step, tool):
        self.iterations += 1
        if self.iterations > self.max_iterations:
            raise RuntimeError(
                "Max iterations exceeded")
        tokens_for_step = 2000
        if (self.tokens_used + tokens_for_step
            > self.max_tokens):
            raise RuntimeError(
                "Token budget exceeded")
        self.tokens_used += tokens_for_step
        return execute(tool)
```

Benefits

- Prevents runaway costs
- Simple to implement
- Protects against basic loops

Limitations

- Hard limits interrupt valid workflows
- No pattern detection before limit hit
- Retry = duplicate work (not idempotent)
- No adaptive allocation

When to use: Simple linear workflows (1–3 steps), low-stakes applications.

Advanced: Loop Detection + Adaptive Budgeting

```
class AdvancedWorkflowExecutor:
    def execute_step(self, step, tool, params):
        # 1. Fail fast on loop pattern
        if loop_detector.is_loop(tool, params):
            raise RuntimeError("Loop detected")
        # 2. Idempotency – no duplicate work
        if in_cache(step, tool, params):
            return cached_result
        # 3. Reserve tokens for future steps
        if not budget.can_execute(estimated):
            raise RuntimeError("Budget exceeded")
        # 4. Execute and cache
        result = execute(tool, params)
        cache[key] = result
        return result
```

Loop Detection

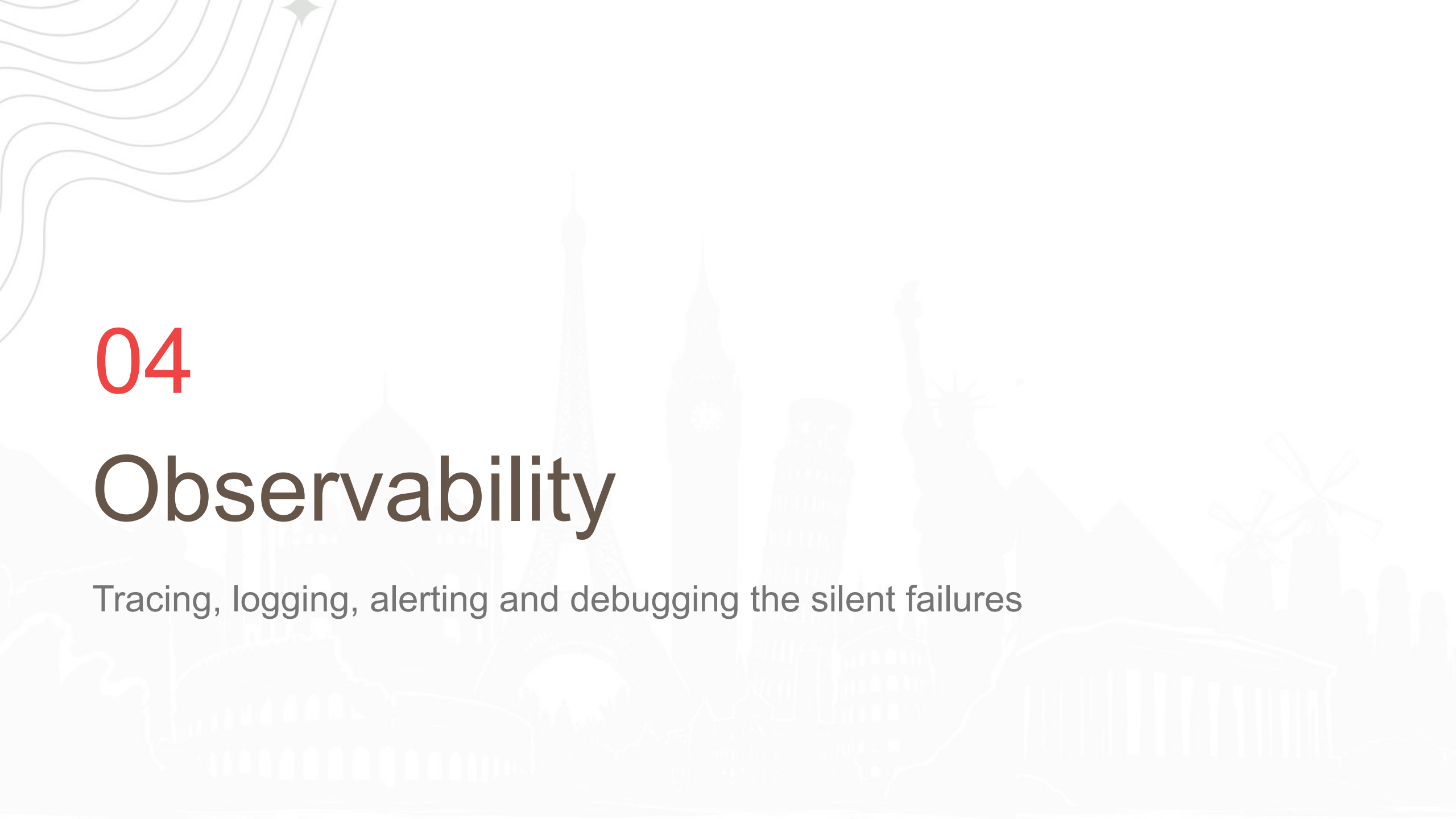
Pattern matching on call history ($A \rightarrow B \rightarrow A \rightarrow B$). Fails fast and doesn't waste budget before stopping.

Adaptive Budgeting

Reserves tokens for remaining steps dynamically. Prevents mid-workflow failures on step 4 of 5.

Idempotency

Cached results mean retries are safe. No duplicate API calls or side effects on failure recovery.



04

Observability

Tracing, logging, alerting and debugging the silent failures

Problem 4: The Agent With No Logs

```
# Your logs
INFO Request received
ERROR Error: validation failed

# Your code
logger.info("Request received")
# ... agent runs (black box) ...
logger.error("Error: validation failed")
```

Questions You Cannot Answer

- Which ticket was being processed?
- What prompt was sent to the model?
- How many tokens were consumed?
- Which tools were called, in what order?
- Is it safe to retry?

What to Log, What to Alert On, How to Debug

Log Everything	Alert On	Debug Clues
Trace IDs, tool calls, params, tokens, latency, model responses	Hallucination rate > threshold, loop patterns, budget exhaustion	Full prompt + response pairs, tool call sequence, state diffs

Basic Solution: Structured JSON Logging

```
trace = {
  "timestamp": datetime.utcnow().isoformat(),
  "trace_id": str(uuid4()),
  "agent_id": "agent-001",
  "event_type": "tool_call",
  "tool_name": "search_web",
  "parameters": {"query": "market trends"},
  "result": {"results": [...]},
  "duration_ms": 342.5,
}
logger.info(json.dumps(trace))
```

Example Queries

```
# All tool calls in a trace
jq '.[[] | select(.trace_id=="123")'
# Total duration
jq '.[[] | .duration_ms] | add'
# Failed validations
grep '"event_type":"error"' logs.json
```

Benefits

- Machine-readable (jq/grep)
- Full execution context captured
- Trace ID links related events

Limitations

- Flat structure
- Single service only
- No standard visualisation

When to use: Single-service apps, fewer than 1M requests/month.

Advanced: OpenTelemetry Tracing

```
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span(
    "agent.execute") as root:
    root.set_attribute("agent.task", task)

    with tracer.start_as_current_span(
        "agent.plan"):
        plan = create_plan(task)

        with tracer.start_as_current_span(
            "agent.tools"):
            for tool in plan.tools:
                with tracer.start_as_current_span(
                    f"tool.{tool}"):
                    result = execute_tool(tool)

    with tracer.start_as_current_span(
        "agent.respond"):
        return generate_response(results)
```

Span Hierarchy

```
agent.execute      (1250ms)
├── agent.plan      ( 145ms)
├── agent.tools      ( 980ms)
│   ├── tool.search ( 342ms)
│   ├── tool.analyze ( 521ms)
│   └── tool.summarize ( 117ms)
└── agent.respond   ( 125ms)
```

Works Across Frameworks

- LangGraph: trace state transitions + tool calls
- AutoGen: trace agent-to-agent messages
- CrewAI: trace task delegation chains

Key Takeaways

1

Classify failures by root cause

Tool misuse, hallucination, context drift, loops, each has different solutions. Don't treat agent failures as a single category.

2

Graduate from Basic → Advanced as you scale

Start with schemas, LLM judge, iteration limits, JSON logs. Don't over-engineer day one. Evolve when metrics demand it.

3

Every advanced pattern has a cost/benefit tradeoff

Semantic entropy: 5× cost → 95% accuracy. Idempotent execution: +20% latency → safe retries. Let data drive decisions.

4

Observability from day one

Trace IDs, structured logs, span hierarchies. If you can't see it, you can't fix it. Don't bolt this on later.

Thank you!

Sepehr Samadi

Product Owner & Senior Software Engineer

samadisepehr@gmail.com

linkedin.com/in/sepehr-samadi

github.com/sepehr1998

AgentCon Rome · May 15, 2026



Code Link

Let's Connect



Scan Me